C 8205 – Interpreter-Projekt: Überblick

Quelle: C8205-Interpreter-Projektueberblick.md

Was hier entsteht und wie die Einzeldokumente zusammenhängen: Ziel und Einordnung, Zwei-Wege-Architektur (Python-/Claude-Interpreter), Teilprojekt Assembler, virtuelle Hardware, Testkonzept, Dokumenten- und PDF-Landkarte.

Layout gemäß C8205-Interpreter-PDF.md (Gestaltungsrichtlinie).

C8205-Interpreter: Projektüberblick

Dieses Dokument richtet sich an Menschen, die das Projekt nicht im Detail verfolgt haben, und beschreibt zusammenhängend, was hier entsteht und wie die Einzeldokumente zueinander stehen. Es ersetzt keines der Fachdokumente, sondern führt in sie ein und verweist auf sie.

1. Ziel und Einordnung

Ziel des Projekts ist die **Simulation der Befehlsverarbeitung** des Cellatron C 8205, eines 1968 in der DDR gebauten Kleinrechenautomaten. Der C 8205 ist ein Einadreßautomat mit vergleichsweise geringem technischen Aufwand (rund 400 Transistoren, 2000 Dioden in der Zentraleinheit) und für einen Kleinrechner hoher Operationsgeschwindigkeit (bis zu rund 2000 verdrahtete Operationen je Sekunde bei zeitoptimaler Programmierung). Anstelle eines RAM-Speichers verwendet er eine Magnettrommel: Zugriffe auf einzelne Speicherzellen sind dadurch nicht beliebig schnell möglich, sondern hängen davon ab, wann die gewünschte Stelle der Trommel unter dem Lese-/Schreibkopf ansteht. Bedient wird der Rechner nicht über einen Bildschirm, sondern über ein Bedienpult mit Tasten, Schaltern und Lampen; Programme und Daten werden über Lochstreifen ein- und ausgegeben.

Die Simulation rekonstruiert diesen Rechner in zwei Schritten: Zunächst wird die Elektronik des Befehlsablaufs (die "vier Phasen" der Befehlsabarbeitung, phasenweise als Knotenautomat mit rund 80 Knoten beschrieben) aus Originalunterlagen so genau wie möglich als Regelwerk rekonstruiert. Darauf aufbauend wird dieses Regelwerk **interpretativ** ausgeführt — wortlautgetreu, ohne stillschweigende Vereinfachung: Ist eine Stelle des Regelwerks unklar, wird das als offener Punkt gemeldet statt eigenmächtig "repariert" ("laut scheitern"-Prinzip). Alle Zeitabhängigkeiten der realen Hardware (Wortzeit, Wartezeit der Trommel) werden für die Simulation auf 0 gesetzt: Die zeitliche Reihenfolge der Ereignisse bleibt erhalten, es entstehen aber keine echten Wartezeiten.

Das Gesamtprojekt gliedert sich dafür in zwei Teilprojekte mit unterschiedlichem Zweck: den **Interpreter** (Python, Wurzel des Projektverzeichnisses; Abschnitt 2) führt Maschinencode gegen das rekonstruierte Regelwerk aus, und den **Assembler** (C, Unterverzeichnis Assembler/; Abschnitt 3) übersetzt lesbaren Assembler-Quelltext in genau diesen Maschinencode. Beide sind eigenständige Werkzeuge, die über den von ihnen erzeugten bzw. verarbeiteten Objektcode zusammenhängen, aber unabhängig voneinander entwickelt werden.

2. Zwei-Wege-Architektur: Python-Interpreter und Claude-Interpreter

Ein und dasselbe Testformat (Abschnitt 6) kann auf zwei unabhängigen Wegen ausgeführt werden, die absichtlich **auf gleiche Art und Weise** arbeiten — derselbe Knotengraph, dieselbe Testdatei, dasselbe Protokollformat:

- **Python-Interpreter** (c8205_interp.py): führt den Knotengraphen deterministisch und automatisiert aus. Schnell und reproduzierbar, aber ohne Rückfragemöglichkeit bei unklaren Regelwerk-Stellen. Er liest den Knotengraphen und die Variablentabelle T2 zur Laufzeit direkt aus den Regelwerk-Dateien (nicht fest einprogrammiert); nur die Wirkung jeder einzelnen ANWEISUNG ist von Hand als Python-Funktion hinterlegt. Fehlt zu einer ANWEISUNG eine Funktion, meldet der Interpreter das als Stub statt zu raten.

- **Claude-Interpreter**: Eine KI erhält das komplette Regelwerk als Kontext sowie dieselbe Testdatei und arbeitet den Knotengraphen manuell, Knoten für Knoten, ab. Langsamer und nicht deterministisch reproduzierbar (abhängig von Modell und Sitzung), erlaubt dafür aber Rückfragen und Erläuterungen während der Abarbeitung.

Rollenverteilung: Während der Python-Interpreter noch nicht vollständig und stubfrei ist, dient der Claude-Interpreter als unabhängige Gegenprobe — weichen die Ergebnisse beider Wege bei einem Test voneinander ab, liegt ein Fehler in einem der beiden vor (oder eine tatsächlich unklare Stelle im Regelwerk). Im späteren Normalbetrieb übernimmt der Python-Interpreter die laufenden Testläufe; der Claude-Interpreter kommt dann gezielt bei Rückfragen sowie beim Durchspielen neuer oder geänderter Befehle zum Einsatz, bevor diese in c8205_interp.py übernommen werden. Diese Aufgabenteilung gilt modellneutral für jede ausführende KI, nicht nur für Claude.

3. Teilprojekt Assembler

Der Assembler c82ass (Cross-Assembler für C8205/C8205Z, Version 0.2, in C geschrieben, läuft unter Linux und Windows 10) ist ein eigenständiges Werkzeug, kein Interpreter oder Simulator: Er übersetzt lesbaren Assembler-Quelltext (.asm) in ladefähigen Objektcode (K1K2-Textformat) für das "Kleine Eingabeprogramm" des C 8205, denselben Objektcode, wie ihn der Interpreter (Abschnitt 2) als Trommeldatei ausführt.

- **Drei Pässe:** Pass 1 ermittelt die Literaltabelle, Pass 2 die Symboltabelle (beide im .log), Pass 3 erzeugt den modifizierten Quelltext mit aufgelösten Literalen (.mqt) und daraus die Kennzeichenbefehle im Objektfile (.obj).
- **Sprache:** Maschinenbefehle (3 Oktalziffern) gemischt mit Assembleranweisungen zur Datendefinition (db/dbn Bereich frei/auf Null, dw Wert, da Adresse, dt/dto/dtm Text als R300-Zeichenfolge, dz Sonderzeichen, dp Programmstart/-ende, equ Adresszuweisung) sowie start/end als Quelltextklammer. Adresssymbole (bis 32 Zeichen, mit Doppelpunkt abgeschlossen) und Literale (#Werteausdruck bzw. @Adressausdruck, vom Assembler automatisch auf einem eigenen Speicherplatz abgelegt) ergänzen die Sprache.
- **Makro-Präprozessor:** MACRO/MEND-Blöcke mit Parameterersetzung ($1, $2, …), bedingter Codegenerierung ($IF/$ELIF/$ENDIF mit bis zu vier über |/& verknüpften Vergleichsblöcken) und generischen, mehrfach verwendbaren Labels über einen %-Platzhalter, der bei jedem Aufruf durch eine fortlaufende Nummer ersetzt wird.
- **Erzeugte Dateien** je .asm-Quelle: .log (Tabellen), .obj (Objektcode), .mqo/.mqt (aufbereiteter Quelltext), .sym (Symboltabelle, Adresse↔Symbol). Weitere in der Assemblerdokumentation beschriebene Endungen (.l82) und Werkzeuge (c82load, c82dump, c82simi, C8205Simulator) sind im aktuellen Projektbaum nicht vorhanden (siehe Abschnitt 9, offene Punkte).

5. Virtuelle Hardware im Überblick

Der reale C 8205 (ECHT) besteht aus Stromversorgung, Zentraleinheit (Rechenwerk und Leitwerk) und Peripherie: Magnettrommelspeicher, Bedienpult mit Tastatur/Lampenfeld/Bahnsperrenschaltern, zwei Lochbandlesern, einem Lochbandstanzer und einem Schreibwerk für alphanumerische Ein-/Ausgabe. Für die Simulation wird davon **virtualisiert**, was ein Testlauf am Bildschirm bzw. in Dateien braucht — mit denselben Kanal- und Adresszuordnungen wie beim Original, aber ohne mechanische Bauteile:

- **Bedienpult:** Tasten (E, G, M, Ü, ST u. a.) und Schalter wirken als Tastenfolge auf Register und Flags, statt tatsächlich gedrückt zu werden; Lampen werden durch die protokollierten Flag-Werte ersetzt.
- **Magnettrommel:** statt der physischen Trommel ein adressierter Speicherausschnitt, der einem Test als Textdatei (Adresse:Wert je Zeile, oktal) vorab mitgegeben wird.
- **Lochbandleser (2, Kanäle K1/K2):** je Leser eine virtuelle Streifendatei; ist keine angegeben und fordert das Programm dennoch eine Eingabe an, bleibt der Lauf mit gesetztem Flag E stehen wie beim Original ohne eingelegten Streifen — kein Fehler.
- **Lochbandstanzer (Kanal K1):** Ausgaben werden zeilenweise binär (Spuren w v u p 8 4 2 1) in eine Ausgabedatei geschrieben, statt real gestanzt zu werden.
- **Schreibwerk (Kanal K0/K2):** zeichenweise Textausgabe in eine Datei, mit WRZL (Wagenrücklauf Zeilenvorschub) als Zeilenendekennung, statt auf eine Schreibmaschine.

Jede virtuelle Datei wird nur angelegt, wenn der jeweilige Test das Gerät tatsächlich benutzt, und am Testende zum Download bereitgestellt.

6. Testkonzept: Trommeldatei, Tastenfolge, Testmarken, automatische Prüfung

Jeder simulierte "Rechnerlauf" ist als **Test** definiert, unabhängig davon, ob er anschließend vom Python- oder vom Claude-Interpreter ausgeführt wird. Ein Test besteht aus:

- **Trommeldatei:** der für den Test relevante Ausschnitt des Speichers, zeilenweise als Adresse: Wert (beide oktal), mit Kommentaren (#) und Leerzeilen.
- **Tastenfolge:** definiert den Ausgangszustand von Registern und Flags vor Beginn der Befehlsschleife — welche Tasten/Schalter in welcher Reihenfolge betätigt werden (z. B. H G R für einen regulären Start), einschließlich der jeweiligen Wirkung auf Register und Flags.
- **Testmarken** für die Auswertung, ohne Entsprechung in der realen Elektronik:
- #DOK-xxxx <Text> löst bei Erreichen des Leerbefehls 164 an Adresse xxxx eine Diagnoseausgabe aus (Text, Register BZ/AC, Flags FE/FÜ/BS/US/E/A).
- #SPEI-xxxx ssss-eeee gibt zusätzlich einen Speicherbereich aus.
- #SPUR-xxxx aaaa gibt zusätzlich eine ganze Trommelspur in physischer Zugriffsreihenfolge aus.
- #PROT STUFE=n (0/1/2) steuert die Protokolltiefe, bis hin zu einer vollständigen, Knoten-für-Knoten protokollierten Protokolldatei bei STUFE=2.

Automatische Prüfbarkeit: Weil derselbe Test mit demselben Format deterministisch (Python) oder als Gegenprobe (Claude) ausgeführt werden kann, lassen sich für jeden Test **Sollwerte** festhalten — Endzustand von Register/Flags, erwartete Speicherinhalte, erwartete Ausgaben. Damit wird jeder Testlauf zu einem Regressionstest: Ein neuer Lauf wird automatisch gegen die zuletzt bestätigten Sollwerte verglichen, ohne dass ein Mensch das Protokoll erneut Zeile für Zeile lesen muss. Referenzläufe dieser Art sind bereits für mehrere Tests festgehalten (u. a. Tests 0004 bis 0021, s1-R1 bis s4-spur).

Ergänzend dazu prüft c8205_befehlssweep.py systematisch alle 512 möglichen Operationscodes (000–777 oktal) gegen den Knotengraphen: Für jeden Code wird ein Minimalprogramm gebaut und direkt über dieselbe Knotengraph-Engine wie c8205_interp.py ausgeführt. Das prüft nicht die numerische Richtigkeit jeder Formel — dafür bleiben die inhaltlichen Einzeltests zuständig —, sondern ob für jeden Code überhaupt eine ANWEISUNG-Implementierung vorhanden ist und der Automat sauber terminiert (kein Stub, kein Sackgassen-Knoten, keine erkennbare Endlosschleife): ein automatisierter Vollständigkeits-Regressionstest zusätzlich zu den inhaltlichen Einzeltests.

7. Dokumentenlandkarte (.md-Dateien und Code)

Datei	Rolle
C8205-Interpreter-Überblick_und_Gesamtaufbau.md	Wurzeldokument: Einordnung, Geräteaufbau, Rechen-/Leitwerk, die vier Phasen der Befehlsabarbeitung, Speicher-Timing und Leistungswerte
C8205-Interpreter-Magnettrommel.md	Speicherorganisation der Trommel (128 Spuren × 32 Sektoren), Interleaving/physische vs. logische Sektoradresse, Bahnsperrenschalter
C8205-Interpreter-Bedienpult_mit_Schaltern.md	Bedienelemente: Tasten, Schalter, Kanalanwahl, Wirkung auf Register/Flags
C8205-Interpreter-Lochstreifenleser.md	Hardwarebeschreibung Lochbandleser, -stanzer und Schreibwerk (Kanäle, Ablauf einer Zeichen-Ein-/Ausgabe)
C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten-mod4.md	Register (AC, BZ, AR, OR, BR), Flags, Maschinenwort- und Befehlswortaufbau (Bitfelder)
C8205-Interpreter-Eingänge_der_Knoten.md	Der Knotengraph selbst: alle Knoten (Konnektor/Phasenzuordnung/Anweisung/Test), Ein-/Ausgänge, Verzweigungen
C8205-Interpreter-Beschreibung_der_Knoten.md	Variablentabelle T2: Bedeutung jeder ANWEISUNG-nnn, die im Knotengraph referenziert wird
C8205-Interpreter-Befehlsliste-Zwischenstand.md	Generische Befehlsbeschreibung nach B/A/Z sowie die vollständige, nach Oktalcode geordnete Einzelbefehlsübersicht (372 Befehle)
C8205-Interpreter-Kodetabellen.md	Zeichenkodetabellen: Lochstreifenkode (Tabelle 1), R300-Lochkartenkode (Tabelle 2), Fernschreibkode CCITT-2 (Tabelle 3)
C8205-Interpreter-Durchführung_eines_Tests.md	Das Testkonzept selbst (Abschnitt 6 dieses Dokuments): Trommeldatei, Tastenfolge, Testmarken, virtuelle Geräte, Protokollierung, Python-/Claude-Interpreter
C8205-Interpreter-Python_Interpreter.md	Bedienung von c8205_interp.py: Installation, Aufruf, Schalter, Fehlermeldungen, Beispiele; Vorgaben für eine noch zu erzeugende Bedienungsanleitung

Datei	Rolle
baz-wirkungstabelle.md	Cache bereits hergeleiteter Knotenpfade/Formeln je getestetem BAZ-Wert, um wiederholte Herleitung zu vermeiden
C8205-Projektstruktur.md	Wurzeldokument der Verzeichnisstruktur: welches Verzeichnis/welche Endung welchen Zweck hat, für Interpreter und Assembler
C8205_Assembler_Dokumentation.md	Durchsuchbare Textableitung der maßgeblichen C8205_Assembler_Dokumentation.odt: Sprache, Anweisungen, Makro-Präprozessor, Beispiel

Alle Dateien werden zwischen Sitzungen erneut hochgeladen, da sie sich auch außerhalb einzelner Gespräche weiterentwickeln.

Code-Dateien (kein Regelwerk, sondern dessen Umsetzung bzw. Werkzeuge dafür):

Datei	Rolle
c8205_interp.py	Der Python-Interpreter selbst (Abschnitt 2): liest Knotengraph und T2 zur Laufzeit aus den .md-Dateien, führt Tests aus
c8205_befehlssweep.py	Vollständigkeitstest über alle 512 Operationscodes (Abschnitt 6)
Assembler/Sourcecode/ (C-Dateien)	C-Quellcode des Assemblers c82ass selbst (Abschnitt 3)

8. PDF-Dokumente im Überblick

Zusätzlich zu den .md-Quellen entstehen für einzelne Themen gut lesbare, A4-taugliche PDF-Ausgaben nach einheitlicher Gestaltungsrichtlinie (C8205-Interpreter-PDF.md):

PDF	Inhalt	Quelle
C8205-Knotengraph_A4.pdf	Der komplette Knotengraph als Diagramm, auf A4-Kacheln aufgeteilt	Eingänge_der_Knoten.md
C8205-Einzelbefehlsliste_A4.pdf	Generische Befehlsbeschreibung und vollständige Einzelbefehlsübersicht	Befehlsliste-Zwischenstand.md
C8205-Kodetabellen_A4.pdf	Die drei Zeichenkodetabellen, Lochbild-Spalten als ●/○-Grafik	Kodetabellen.md
C8205-Interpreter-PDF.md	Gestaltungsrichtlinie selbst (kein eigenständiges PDF, aber Grundlage aller anderen)	—

Geplant, noch nicht erstellt: eine Bedienungsanleitung für den Python-Interpreter (c8205_interp.py), deren Gliederung, Zielgruppe und Pflichtinhalte in Python_Interpreter.md, Abschnitt 14, bereits festgelegt sind.

9. Stand und offene Punkte

Einige Punkte sind in den Fachdokumenten selbst als noch ungeklärt vermerkt und absichtlich nicht stillschweigend entschieden:

- Genaue Kopplung zwischen Schalter/Flag SW und der programmierten Schreibwerkausgabe über Kanal K2 (Durchführung_eines_Tests.md, Abschnitt 6.3.1).
- Bedeutung der Pult-Tasten B und D — kommen weder in den Kodetabellen noch als Sonderzeichen im Fernschreibkode vor (Kodetabellen.md, Abschnitt 2.4).
- Ob die FS-Kode-Zeile "IS" (Irrung Satz) mit der gleichnamigen Zeile in Tabelle 1 identisch ist (Kodetabellen.md, Abschnitt 3.3).
- Punktmuster der FS-Kode-Spalte "Kanal 54.321" noch nicht bitgenau je Zeile erfasst (Kodetabellen.md, Abschnitt 3.3).
- Brücke Assembler → Interpreter: Wie ein Assembler-Ergebnis (.obj) zu einem Interpreter-Testauftrag wird, ist nirgends beschrieben (Projektstruktur.md, Abschnitt 4).
- Zweck und erzeugende Bedingung der Assembler-Endungen .dmp, .mac, .test ungeklärt; ebenso, ob die in der Assemblerdokumentation genannten Endungen .mqt und .l82 entfallen sind oder nur bei bestimmten Läufen entstehen (Projektstruktur.md, Abschnitt 4).
- Die in der Assemblerdokumentation erwähnten Werkzeuge c82load, c82dump, c82simi und C8205Simulator liegen im aktuellen Projektbaum nicht vor (Projektstruktur.md, Abschnitt 4).

Diese Liste erhebt keinen Anspruch auf Vollständigkeit; maßgeblich bleiben die "offen"-Vermerke in den jeweiligen Fachdokumenten.